The QExpress Framework for Quantum Compilation

Mudi Yang

A thesis presented for the degree of
Master of Science

COLUMBIA
ENGINEERING

Department of Computer Science
Fu Foundation School of Engineering and Applied Science
Columbia University
Spring 2026

Dedication

To Mom, Dad, and Deniz, for your love generously given.
To all who helped me, for your advice and insight.
To good friends, for cherished, unserious moments.

Acknowledgements

I want to thank Professor Henry Yuen for advising my degree and making it possible for me to pursue the MS thesis track. I want to thank Professor Dan Rubenstein for overseeing the QExpress project and for advising this work. This project would never have started without the both of you. I am lucky to have you as advisors.

To the student members of the QExpress project: Kahlil Dozier, Shivalee Shah, and Christine Li, I express my gratitude for listening to my rambles and helping me turn them into something concrete. You helped and continue to help me refine my thinking. This we did together.

I would like to thank the members of my thesis committee: Henry Yuen, Dan Rubenstein, and Yunong Shi for your time, patience, and excellent questions. Your feedback has been my north star.

Finally, I would like to thank the University and the Department. The opportunity to be a student has been a delight.

The QExpress Framework for Quantum Compilation

Mudi Yang

Abstract

Quantum compilers are commonly engineered as a hardware specific stack because qubit placement, gate scheduling, and routing depend on physical constraints that differ across neutral atom, trapped ion, and superconducting systems [1, 2, 3, 4, 5]. This tight coupling raises implementation cost and limits portability. Prior generalized efforts built on classical infrastructure (for example, LLVM-based approaches) demonstrate feasibility but still leave a substantial engineering barrier for rapid experimentation [6].

This thesis presents **QExpress**, a lightweight, Python-based compiler framework whose main contributions are threefold. First, it introduces **QExpressions**, an intermediate representation that maps circuit logic onto a virtual machine model suitable for downstream compilation. Second, it provides portable analysis and optimization pass infrastructure so transformations can be reused across multiple hardware targets rather than reimplemented per architecture. Third, it defines reusable abstractions for hardware details, primarily connectivity and state tracking, so architecture specific extensions remain focused on hardware only responsibilities while core compilation logic is shared.

To evaluate modularity and reuse, three hardware extensions are implemented for neutral atom, trapped ion, and superconducting targets. The implementation data show that most complexity is centralized in the reusable framework, while each hardware extension remains compact. A generalized unblocking algorithm is further shown to apply across architecture specific phenomena (trap capacity limits, Rydberg blockading, and cross-talk), illustrating cross-architecture transfer of compiler techniques.

These results suggest that QExpress can reduce the developer effort necessary for building new quantum compilers while retaining hardware aware optimization opportunities.

Contents

1 Introduction **6**

2 Current Compilers **8**
2.1 Neutral Atom . . . 8
2.2 Trapped Ion . . . 9
2.3 Superconducting . . . 9

3 Framework Overview **11**
3.1 Circuits to Hardware . . . 11
3.2 Front End . . . 12
3.2.1 List Operations . . . 12
3.2.2 Reorder Operations . . . 13
3.2.3 The QExpress Virtual Machine . . . 14
3.2.4 Generate QExpressions . . . 14
3.3 Back End . . . 17
3.3.1 Initial Qubit Placement . . . 17
3.3.2 Qubit Routing . . . 18
3.4 Hardware Specific Extensions . . . 20
3.4.1 Adjacency List Creation . . . 21
3.4.2 Process Path . . . 23
3.4.3 Hardware Code Realization . . . 24

4 Compiler Overview **26**
4.1 A QExpress Compiler . . . 26
4.2 The Pass Manager . . . 27
4.3 Compiler Usage . . . 28

5 Results and Further Work **29**
5.1 Framework Evaluation . . . 29
5.1.1 Unblocking . . . 30
5.2 Further Work . . . 31

6 Conclusion **32**

A QExpress Implementation Details **33**
A.1 QExpress Code Overview . . . 33
A.1.1 Front End . . . 33
A.1.2 Back End . . . 33
A.1.3 Optimization & Analysis . . . 34
A.2 Examples . . . 34
A.2.1 Neutral Atom Example . . . 35
A.2.2 Trapped Ion Example . . . 36
A.2.3 Superconducting Example . . . 38

Chapter 1

Introduction

Programming a quantum computer is a difficult task. This difficulty arises from two primary factors. First, most current quantum computers are effectively physics experiments rather than standardized computing platforms. Each system is implemented using distinct physical mechanisms, control electronics, and operational constraints. As a result, the process of translating a sequence of quantum logic gates, a quantum circuit, into executable hardware instructions typically requires a bespoke compiler tailored to the specific device [1, 2, 3, 7, 8]. Second, designing and implementing a compiler is itself a challenging problem. Many tasks involved in quantum compilation, such as optimal qubit mapping or circuit scheduling, are believed to be NP-hard [1, 9]. Combined with the uniqueness of individual quantum systems, this often results in compiler infrastructures being developed from scratch for each platform [1, 10, 5]. The consequence of these factors is that programming any given quantum computer can be a complex and time consuming endeavor.

This work is motivated by the observation that classical computing faced a similar problem before the development of modern compiler infrastructures. In classical compilation, reusable frameworks such as LLVM provide a structured environment in which compiler components, including intermediate representations, optimization passes, and back end code generation, can be developed and reused across many target architectures. Such frameworks reduce the engineering effort required to build new compilers and allow researchers to focus on optimization and architectural innovation rather than reimplementing common infrastructure.

LLVM itself was not adopted directly in this work for several reasons. The system presents a significant barrier to entry for many researchers due to its complexity and implementation in C++. More importantly, LLVM was designed primarily for classical, von Neumann architecture workloads centered around CPU-based execution. Although LLVM has been successfully extended to support highly parallel architectures such as GPUs, its abstractions remain rooted in classical computing assumptions that do not map cleanly onto many aspects of quantum hardware. Previous work by [6] attempted to build quantum compilation infrastructure on top of LLVM, but the resulting framework has not seen widespread adoption within the quantum computing community. This limited adoption is likely influenced by the mismatch between LLVM's design goals and the needs of quantum compilation.

In response to these challenges, this work explores the development of a lightweight, quantum specific compiler framework intended to simplify the construction of compilers for diverse quantum hardware systems. The framework is implemented in

Python 3 and aims to lower the barrier to experimentation while encouraging reuse of common compilation components. By providing shared infrastructure for representing quantum programs, organizing optimization passes, and generating hardware instructions, the framework seeks to reduce the need for entirely bespoke compiler implementations for each new quantum device.

At the core of this framework are **QExpressions**, an intermediate representation designed to capture the structure of quantum programs while remaining sufficiently flexible to support multiple hardware targets. The framework also introduces mechanisms for managing analysis and optimization passes, enabling researchers to develop transformations that can be applied consistently across different compilation targets. Together, these components form the **QExpress Compiler Framework**, which is designed to facilitate rapid prototyping of quantum compilers while still allowing developers to incorporate hardware-specific behavior when necessary.

The goal of this work is therefore not only to demonstrate a particular compiler implementation, but also to present a reusable infrastructure that can support ongoing research in quantum computation. By lowering the engineering overhead associated with compiler construction, the framework aims to enable researchers to focus more directly on algorithmic improvements, hardware aware optimizations, and the exploration of new quantum computing architectures.

Chapter 2

Current Compilers

This section reviews three state-of-the-art custom quantum compilers, one each for neutral atom, trapped ion, and superconducting architectures [2, 3, 9]. Each implementation is strongly hardware dependent, with compilation strategies shaped by architecture specific constraints on connectivity, movement, and interaction primitives. At the same time, these approaches show similarities in algorithmic structure, especially dependency graph based scheduling and heuristic routing decisions [1]. To motivate the compiler discussion, this section first gives a short hardware overview, then examines representative compiler designs for each architecture.

Neutral atom: Qubits are individual atoms held in optical tweezers, with static trap arrays typically generated by spatial light modulators (SLMs) and mobile trap arrays generated by acousto-optic deflectors (AODs) [2]. Two qubit interactions are enabled by moving selected atoms into proximity (within Rydberg radius) and applying global Rydberg excitation. Because atoms are explicitly relocated between storage and compute regions, compilers must jointly optimize gate ordering, inter-zone transfer, and constrained AOD motion.

Trapped ion: Qubits are ions confined in electromagnetic traps, where ions within the same trap can be reordered and effectively treated as all-to-all connected for gate execution via shared motional modes [3]. Scaling beyond a single trap requires shuttling ions across a trap network, so compilation must manage both shuttle paths and finite trap capacity constraints.

Superconducting: Qubits are physically stationary circuit elements (for example, Josephson-junction-based transmons) laid out on a fixed hardware coupling graph [9]. Since qubits do not move and two-qubit gates are limited to connected neighbors, compilation strongly depends on initial mapping and SWAP insertion to realize nonlocal interactions.

With this hardware context in place, the remainder of this section reviews representative compilers for neutral atom, trapped ion, and superconducting systems.

2.1 Neutral Atom

PowerMove targets zoned neutral atom systems and treats compilation as a coupled scheduling and movement problem [2]. The method begins with a dependency structure over two qubit interactions and partitions operations into stages of parallelizable gates acting on disjoint qubits. Stage ordering is selected using criteria that account for both immediate execution cost and effects on subsequent stages.

After stage construction, PowerMove plans movement between stages using a continuous routing strategy. Interacting and noninteracting qubits are treated differently so that required interactions are enabled while nonparticipating qubits are repositioned to support future stages. The resulting one qubit movements are grouped into collective moves and scheduled to improve parallel execution while controlling movement distance.

The approach can be viewed as three tightly connected components: stage construction, movement generation for the next stage, and parallel movement scheduling. This coupling is important in zoned hardware because a locally beneficial movement can induce bottlenecks in later stages. Reported evaluation indicates improvements in both fidelity and runtime compared to prior methods, supporting the view that scheduling and movement should be optimized jointly rather than independently.

Overall, the method emphasizes that in neutral atom architectures, gate scheduling and movement are strongly interdependent. Instead of treating routing as a local correction after scheduling, PowerMove repeatedly uses future stage information when selecting present movement actions.

2.2 Trapped Ion

Muzzle the Shuttle compiles for multi trap trapped ion hardware by first organizing circuit operations through a dependency Directed Acyclic Graph (DAG) and then producing an executable gate sequence with parallel layers [3]. Once this sequence is established, shuttling operations are inserted to bring ions into valid interaction locations.

Its central routing heuristic balances two objectives: reducing immediate movement cost and improving readiness for upcoming gates. In practice, candidate shuttle decisions are scored using both distance based criteria and weighted lookahead over future operations. The same principle appears in trap rebalancing; when a trap reaches capacity, ions are relocated to restore routing feasibility while prioritizing placements that are useful for near future interactions.

The compiler alternates between executing currently legal gates, inserting shuttle operations for blocked interactions, and invoking rebalancing when occupancy constraints prevent progress. In this way, trap capacity management is integrated directly into the routing process rather than applied as a postprocessing step. Reported results show strong reductions in shuttle count and corresponding improvements in fidelity across benchmark circuits.

This approach highlights a key trapped ion challenge; routing is not solely a shortest path problem, but also a congestion management problem under finite trap capacity. A competent compiler must therefore address both.

2.3 Superconducting

SABRE addresses compilation on fixed connectivity superconducting devices where two qubit gates are legal only on adjacent hardware qubits [9]. The method employs a bidirectional heuristic search procedure. It begins from a dependency DAG and maintains a “front layer” of currently executable gates. If a gate in the “front layer”

is not hardware legal, SWAP operations are inserted to make the required qubits adjacent.

To keep search costs practical, SABRE limits SWAP candidates to qubits relevant to the current "front layer" and scores each candidate with a heuristic that combines immediate distance reduction and expected impact on near future gates. A decay term is used to trade off circuit depth and total SWAP count by discouraging repeated movement of the same qubits. This lookahead-plus-decay approach reduces decisions that satisfy only the current gate while degrading later routing quality.

SABRE further improves initial mapping through reverse traversal. A forward run produces a final mapping, and that mapping seeds a reverse direction optimization to derive a better starting assignment. This bidirectional strategy allows initial placement and midcircuit routing to reinforce one another while maintaining scalability and solution quality.

As shown above, existing quantum compiler implementations are complex. If each useful quantum computer requires a compiler to be developed from scratch, practical use of that computer must first overcome the cost of compiler construction. This requirement is nontrivial and constitutes a substantial barrier to adoption.

Chapter 3

Framework Overview

3.1 Circuits to Hardware

In the previous chapter where current compilation approaches were discussed, two facts and their consequences are clear. First, current compilers are exceedingly hardware aware. Their ordering, placement, and routing algorithms depend heavily upon hardware details including topology, movement mechanism, and qubit implementation. This results in highly specific implementations for which advancements are restricted in portability across architectures and potentially even within the same architecture class when unique hardware capabilities are involved [11].

Second, current compilers are doing remarkably similar things. All the compilers detailed above and other surveyed compilers attempt to solve the following three problems or a subset thereof: gate ordering, initial qubit placement, and qubit routing. Additionally, these problems are solved in much the same way, if the algorithmic details involving hardware specifics are ignored. Gate ordering is determined by a topological sort and an analysis of special cases where reordering can lead to improvements in qubit fidelity or movement reduction. Initial placement seeks an arrangement to minimize future qubit movement. Routing determines which move from a list of candidates should be executed based on a heuristic balancing distance moved and how well the move sets up future interactions (there are implementations which use a brute-force search over all moves, but these have fallen out of favor due to long compilation times for even simple circuits [1]). In fact, the three compilers detailed in the previous section, representing the state-of-the-art in compilation for each architecture, can all be categorized in this manner.

Therefore, if these problems can be treated in a hardware independent manner, then these implementations could be used on any platform. If a collection of these solutions could be packaged together into a framework, then that framework would be capable of solving most of the difficult problems of the compilation process. To create a bespoke compiler for any quantum computer then, all the programmer would need to do is to extend the framework with some specific hardware details of that computer. This would dramatically reduce the difficulty of implementing a custom compiler for any computer.

This raises a new question: is it possible to do the majority of compilation tasks in a hardware independent manner? The remainder of this will demonstrate that it is possible by example. The implementation details of such a framework will be described in detail, and then that framework will be used to develop three compilers,

one for each of three hypothetical neutral atom, trapped ion, and superconducting systems.

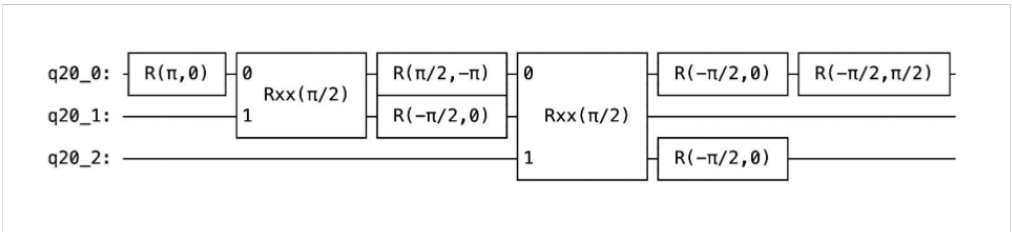
The QExpress framework is organized into two sections. The front end contains implementations to solve hardware agnostic problems, and the back end contains implementations to solve hardware aware problems in a hardware independent manner. An intermediate representation (IR) called QExpressions is used throughout, and a robust optimization manager oversees the use of optimization and transformation passes upon QExpressions.

3.2 Front End

The QExpress Front End provides support for the completion of three main tasks: listing operations, reordering operations, and the generation of the QExpressions IR. It should be immediately clear that these tasks do not rely upon any knowledge of hardware details as they are concerned with interpreting and transforming the logic of a quantum circuit which is inherently hardware independent. Details for each of these tasks are described below.

3.2.1 List Operations

In order to allow for simpler processing of quantum circuit logic, the first step taken is to transform the quantum circuit ingested into a sequence of operations. Each operation corresponds to a quantum gate within the original circuit, and it contains additional information on which qubits are involved as well as the parameters of the gate, as is often found in trapped ion systems. Control and target qubits are specified by the order in which they appear. **Figure 3.1** shows an example circuit and the resultant list of operations generated. This specific circuit will be used as an example occasionally in the rest of this text.



[{'name': 'r', 'params': ['pi', '0'], 'qubits': ['q0']},
{'name': 'rxx', 'params': ['pi/2'], 'qubits': ['q0', 'q1']},
{'name': 'r', 'params': ['pi/2', '-pi'], 'qubits': ['q0']},
{'name': 'r', 'params': ['-pi/2', '0'], 'qubits': ['q1']},
{'name': 'rxx', 'params': ['pi/2'], 'qubits': ['q0', 'q2']},
{'name': 'r', 'params': ['-pi/2', '0'], 'qubits': ['q0']},
{'name': 'r', 'params': ['-pi/2', 'pi/2'], 'qubits': ['q0']},
{'name': 'r', 'params': ['-pi/2', '0'], 'qubits': ['q2']}]

Figure 3.1: Quantum circuit with parameterized gates and corresponding list of operations. The operations each correspond to a quantum gate and contain all the relevant qubit and parameter information necessary for further processing.

3.2.2 Reorder Operations

In theory, there is nothing preventing the operations to be executed in circuit order as originally presented. However, there are often benefits to be gained from a computation perspective if the operations are reorder. A common practice is to reorder so that operations upon the same qubits are done together as much as possible [2, 4, 3, 1]. This reordering scheme has many benefits. In zone based architectures, there is often limited space in the compute zone, and qubits in the compute zone acquire error at an accelerated rate compared to those in storage zones. Therefore, it is beneficial for qubits to spend as little time as possible in the compute zone, and executing all possible operations on a qubit as soon as it is moved into the compute zone aids in this objective. In all architectures, there is often some degree of parallel execution possible, and executing operations in circuit order does not take advantage of this increased potential for computational throughput. As qubit coherence times are a concern on all architectures, identifying opportunities for parallelization is important. Reordering in the manner above allows for identification of separate operation "clusters" which may be executed independently.

The QExpress framework provides robust support for these reordering operations through the construction of operations-based directed acyclic graphs (DAGs). Operations form the nodes, and logical dependencies produce the edges. There are implemented methods for generating topological sorts from the graph as well as identifying parallelization portions. **Figure 3.2** shows the DAG constructed from the example circuit. From this DAG, a topological sort with any desired reordering is created. For this example, no reordering is done, and the sorted order corresponds to the circuit order. The sorted operations are then used in the next step, QExpression generation.

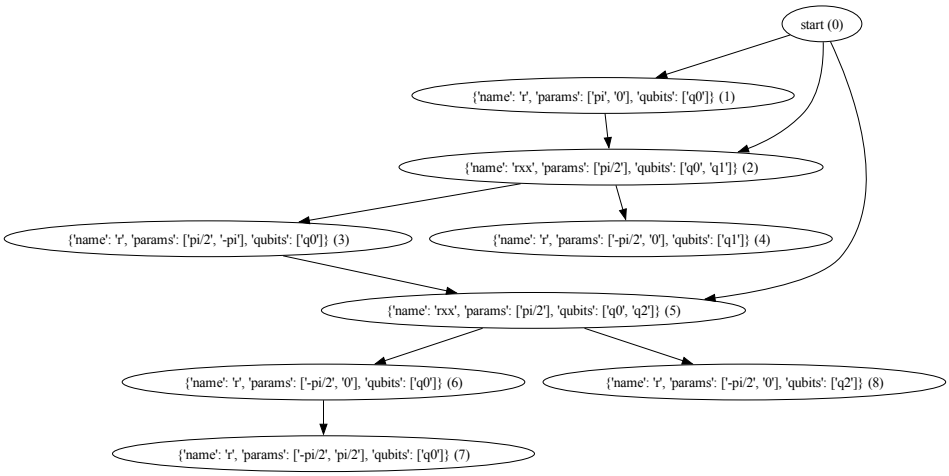


Figure 3.2: DAG constructed from example circuit. Nodes correspond to operations and edges correspond to logical dependencies resulting from order and use of qubits. There are multiple valid topological sorts for this simple circuit, and it is possible that one ordering is more beneficial than another from a computation perspective.

From this step on, the compilation process begins to move from the domain of pure circuit logic to the logic of physical computation. This is done in two stages. First,

the operations are mapped to the QExpress Virtual Machine (QVM), an idealized quantum computer with unrestricted resources and connectivity. These virtual machine operations are in turn eventually mapped to the desired real hardware. In order to better explain the role of the QExpressions IR as well as indicate what they may be use for, a digression on the QVM will be presented.

3.2.3 The QExpress Virtual Machine

The QExpress Virtual Machine serves to anchor circuit level logic with a sense of "locality" without placing restrictions on what may be actualized on real hardware. The virtual machine consist of two zones, a compute zone and a storage zone, each with unlimited space. The zones consist of sites which are locations capable of containing a qubit. Any arbitrary connectivity within and between sites of the two zones can be specified, but the default connectivity is all-to-all within the compute zone and one-to-all from each storage site to all compute sites. Qubits may only be one site at a time, sites may only contain one qubit at a time, and gates can only be applied within the compute zone. Movements are direct. Multiple operations, including movements and gates, may occur simultaneously so long as the operations are independent.

The purpose of the QExpress VM is to provide developers with a framework for thinking about computation optimizations which otherwise would be impossible to do if only circuit level logic is considered. **Figure 3.3** shows a hypothetical snapshot of computation including moves and a gate. The VM allows the developer to reason about when qubits are moved, where they are moved to, and what they are adjacent to among other computation details. Those familiar with the virtual memory abstraction in classical computing may find parallels. Indeed, this representation is largely inspired by its classical counterpart.

QExpression State Tracker (Virtual)

In order to track the state of VM computation, a simple abstraction named the **QExpression State Tracker (QEST)** is implemented to assist development. The QEST consists of three dictionaries mapping the following:

- Qubit to Virtual Site
- Virtual Site to Qubit
- Virtual Site to Zone

These dictionaries are updated upon each operation. This abstraction used in conjunction with a description of site connectivity effectively implements the QExpress VM as all properties are encompassed and any valid computation state can be described. While this current abstraction deals only with virtual sites, an analogue exists for hardware sites and will be described in the **Section 3.3**. Effort will be made to disambiguate between the virtual and hardware versions of the QEST.

3.2.4 Generate QExpressions

The QExpressions IR are programs for the QExpress VM. Therefore, the process of generating these IR is process of converting circuit logic to VM instructions. In order

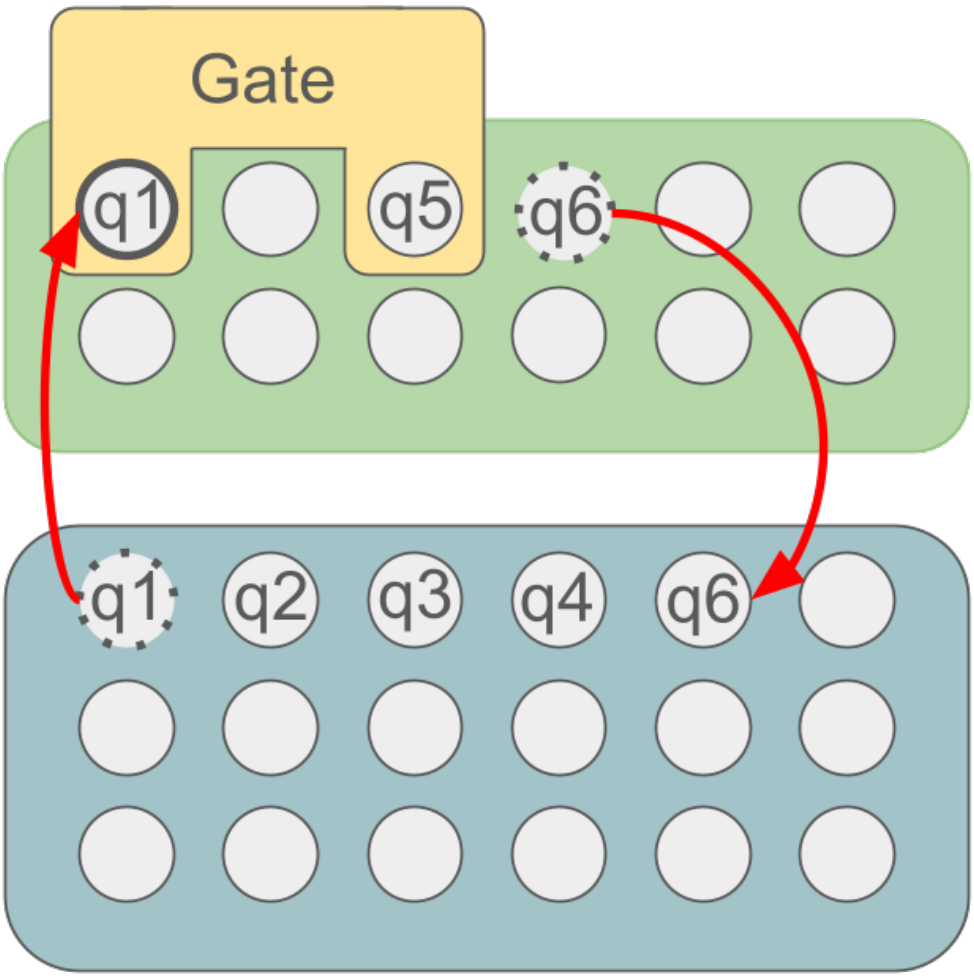


Figure 3.3: A representation of the QExpress VM during computation. The light green zone is the compute zone, and the teal zone is the storage zone. Qubit 1 is being moved from the storage zone to the compute zone while the opposite is done to qubit 6. A gate is applied across qubits 1 and 5. The two movement may occur in parallel while the gate must occur after qubit 1 is moved into the compute zone.

to do this, a few more simple elements must be introduced. First is the `Region`. The region is a container for the entire compute surface, and the aforementioned compute and storage zones are constituent parts of the region. The `OperationZone` corresponds is a generalization of the zone concept, and at declaration time, a list of which operations may be performed in an instance of an `OperationZone` are provided. Therefore, to create separate compute and storage zones, two `OperationZone`s are instantiated, each with the respective `Storage` and `Compute` capabilities provided to the constructor. Other elements such as `site`, `qubit`, `MOVE`, and `operation` correspond directly to virtual sites, qubits, qubit movement, and gates described previously. **Figure 3.4** shows the QExpressions generated corresponding to the example circuit previously described.

Generating these instructions from sorted list of operations generated at the end of the previous step is fairly straightforward. First, the region and contained operation zones are declared. Then a site is created in the storage zone for each qubit in the program. All qubits start in the storage zone and are assigned to

their corresponding site therein. A site is created in the compute zone for each qubit in the program. This arrangement, where the number of sites in the storage and compute zones are both equal to the number of qubits, guarantees that the computation process never runs out of spatial resources, but a user can modify this behavior if desired. Gates are then applied in order with qubits moved to the compute zone as needed and moved back into the storage zone after their last use. If the policy that each qubit is moved into the lowest numbered compute site available and immediately removed after its last use, some compute sites may be unused during the entire computation process. It is then simple to remove all references to these unused sites to produce a minimal required set of computation sites. The QExpress framework provides an optimization pass to do just this.

```
Region = new Region()
OpZone Storage = new OperationZone(Storage)
Region.addOperationZone(Storage)
OpZone Comp = new OperationZone(Comp)
Region.addOperationZone(Comp)
Site s0 = new Site(0)
Storage.addSite(s0)
Qubit q0 = new Qubit(s0)
Site s1 = new Site(1)
Storage.addSite(s1)
Qubit q1 = new Qubit(s1)
Site s2 = new Site(2)
Storage.addSite(s2)
Qubit q2 = new Qubit(s2)
Site s3 = new Site(3)
Compute.addSite(s3)
Site s4 = new Site(4)
Compute.addSite(s4)
Site s5 = new Site(5)
Compute.addSite(s5)
MOVE(s0, s3)
QEXP_VM.applyOperation(sites = [s3], gate_name = 'r', parameters = ['pi', '0'])
MOVE(s1, s4)
QEXP_VM.applyOperation(sites = [['s3', 's4']], gate_name = 'rxx', parameters = ['pi/2'])
QEXP_VM.applyOperation(sites = [s3], gate_name = 'r', parameters = ['pi/2', '-pi'])
QEXP_VM.applyOperation(sites = [s4], gate_name = 'r', parameters = ['-pi/2', '0'])
MOVE(s4, s-1)
MOVE(s2, s5)
QEXP_VM.applyOperation(sites = [['s3', 's5']], gate_name = 'rxx', parameters = ['pi/2'])
QEXP_VM.applyOperation(sites = [s3], gate_name = 'r', parameters = ['-pi/2', '0'])
QEXP_VM.applyOperation(sites = [s3], gate_name = 'r', parameters = ['-pi/2', 'pi/2'])
MOVE(s3, s-1)
QEXP_VM.applyOperation(sites = [s5], gate_name = 'r', parameters = ['-pi/2', '0'])
MOVE(s5, s-1)
```

Figure 3.4: QExpressions representation for the example circuit. This IR is a program for the QExpress VM, and optimizations or analysis related to the computation process can be performed upon it.

Once the QExpressions are generated, a DAG is constructed from the instructions and maintained. This DAG is useful for optimizations, analysis, or compilation algorithms requiring the first, last, and n^{th} use of a qubit and conveniently provides dependency and parallelization information. Additionally, it is preferable to process QExpressions in consecutive, parallelizable batches with relationships between in-

structions easily accessible. Therefore, the QExpressions are passed in DAG form to the back end. **Figure 3.5** shows the DAG constructed for the example circuit.

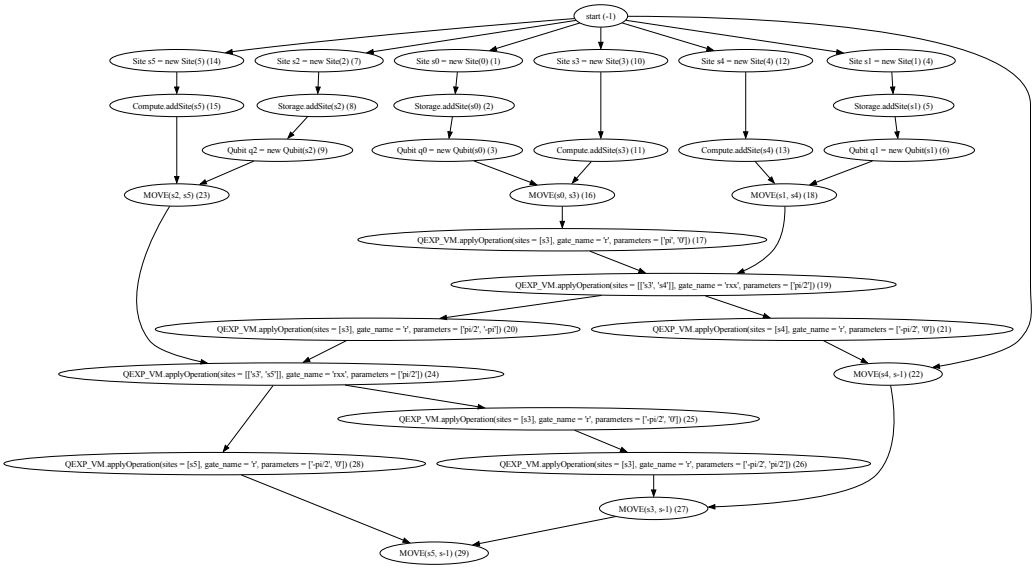


Figure 3.5: DAG built from the QExpressions generated from the example circuit. This graph is used for optimizations, analysis, and further compilation tasks.

3.3 Back End

The QExpress Back End provide support for the completion of two tasks: initial qubit placement and qubit routing during computation. These operations require specific details about the desired target system in order to be executed correctly, and therefore they are distinct from the hardware agnostic front end. Though they are hardware aware, they are still generalizable and therefore the implemented features of the back end are applicable to all architectures. Tasks inseparable from the physical computer are differed to **Section 3.4**.

3.3.1 Initial Qubit Placement

It is obvious that to carry out any computation, there must first be an initial arrangement of qubits. Many options are proposed [1, 9, 2, 12, 13, 14], but there are prevailing trends which may be discerned. Simpler methods include in-order assignment of qubits to compute sites[1] or random assignment. More sophisticated methods include those employed by [9, 4, 2]. Though strategies and implementation details may differ, they all have a goal of finding a starting layout for computation that can reduce the number of qubit movements required to enable future interactions.

The QExpress framework offers implementations for the basic strategies of in-order and random assignment as well as a degree based greedy algorithm. This

[1]Here, the term "compute site" is more general than the definition used within the QExpress framework. It refers to any position capable of holding a qubit for the purpose of interaction or measurement, regardless of wether the underlying architecture delineates a compute zone.

algorithm is reproduced below as **Algorithm 1**. The algorithm tries to satisfy two constraints as much as possible. First, interacting qubits are placed near each other. Second, qubits are matched to starting hardware sites with hardware connectivity degree greater than or equal to their interaction degree.

Algorithm 1 Initial Qubit Placement Heuristic

```
for each DAG node that contains the first use of a qubit q do
    v ← current virtual site of q
    qDeg ← degree of v
    F ← future interaction qubits of q (via DAG traversal)
    if F = ∅ or no qubit in F is mapped to a hardware site then
        choose the first free hardware compute site h such that deg(h) ≥ qDeg
    else
        let q′ be the next qubit in F
        h′ ← hardware site mapped to q′
        A ← free compute sites adjacent to h′
        choose the first h ∈ A such that deg(h) ≥ qDeg
        if no such site exists then
            choose any free compute site
        end if
    end if
end for
```

It should be clear that the only hardware detail necessary for this algorithm is the connectivity of the hardware as represented by an adjacency graph. The creation of this graph from inputs to the compiler varies based on the target architecture and is thus defered to **Section 3.4**. More sophisticated algorithms or optimizations can make use of annotated adjacency lists to achieve goals such as prioritizing use of lower error hardware sites. To encourage such customizations, this and other algorithms within the QExpress framework are purposefully made simple and easily extensible.

3.3.2 Qubit Routing

During the process of computation, it is likely that a qubit will have to be moved in order for an interaction to be executed. In such cases, there are again multiple existing implementations [1, 15, 3, 2, 9, 4] and prevailing trends. Previously popular were SMT solver based searches for optimal routing schemes [16, 1], but this approach leads to long compilation times. Currently popular is a heuristic based search balancing between the total length of the movement and the suitability of the final destination for future interactions. The method of calculating movement distance is obvious, but the determination of suitability is slightly more complex. For a proposed movement path, the score is calculated by taking the final position of the moved qubit and composing a weighted sum of distances from that final location to all other qubits the moved qubit must interact with in the future. Often, these sums have a diminishing weight term so as to weigh interactions further in the future less than those that are more immediate. Routing based on shortest movement distance is a subset of this heuristic in which the balance is skewed entirely in the favor of movement length.

The QExpress framework implements both a heuristic search and the simple shortest move for convenience. The heuristic search is reproduced as **Algorithm 2**. Developers can specify how much they wish for the weight to decay as a function temporal distance through a parameter p for the *futureScore(p)* function.

Algorithm 2 Candidate Path Selection for Two-Qubit Interaction

Require: qubits q_1, q_2 that must interact
$A_1 \leftarrow$ free compute neighbors of q_1
$A_2 \leftarrow$ free compute neighbors of q_2
$P \leftarrow \emptyset$ {set of candidate paths}
for each $a \in A_1 \cup A_2$ **do**
  if $a \in A_1$ **then**
    $p \leftarrow$ BFS path from q_2 to a {move q_2 toward q_1}
  else
    $p \leftarrow$ BFS path from q_1 to a {move q_1 toward q_2}
  end if
  if $p \neq \emptyset$ **then**
    add p to P
  end if
end for
if $P = \emptyset$ **then**
  return None
end if
for each path $p \in P$ **do**
  $proximity(p) \leftarrow |p|$
  $futureScore(p) \leftarrow$ weighted average distance to future interactions
  $totalScore(p) \leftarrow proximity(p) + futureScore(p)$
end for

return $p^* = \arg\min_{p \in P} totalScore(p)$

This algorithm will consider all viable paths that could bring two qubits into adjacency. Thus, it considers moving each of the qubits to any position adjacent to the other qubit. Again, it should be clear that the hardware detail necessary for this algorithm is the connectivity of the hardware as represented by an adjacency graph. In addition to this adjacency graph, the locations of the qubits over the course of the computation must be tracked. Here, the second QEST to be used for mapping between qubits and hardware sites is introduced. This QEST tracks much the same items as the previously described virtual QEST:

- Qubit to Hardware Site
- Hardware Site to Qubit
- Hardware Site to Zone

Figure 3.6 shows the capabilities and dependencies of the QExpress framework. By utilizing the adjacency list representation of hardware connectivity and the two QESTs to track computation state, the QExpress framework is able to generalize

a large portion of quantum compilation logic in a reusable manner such that they are applicable for all architectures of gate based quantum computers. The following sections will deal with extending the framework to create a custom compiler for three hypothetical systems, one each of neutral atom, trapped ion, and superconducting. Until now, the nature of some things such as hardware zones, sites, and movement paths have been kept deliberately vague. They will be made concrete in the examples that follow.

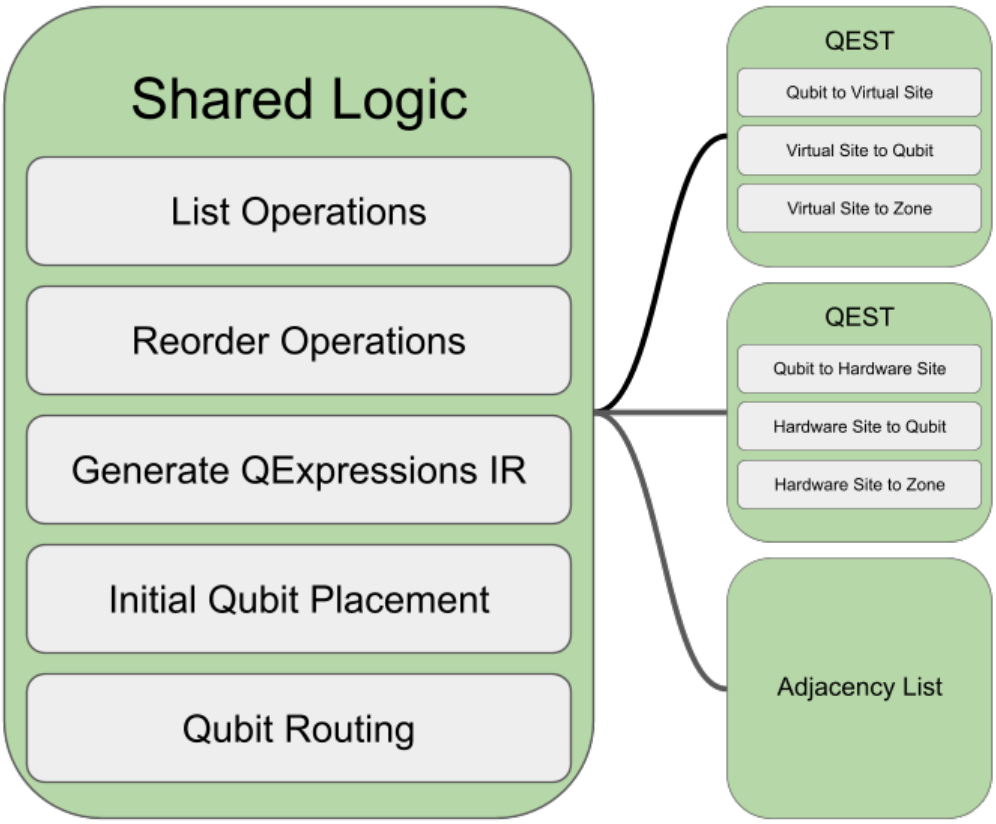


Figure 3.6: A view of the QExpress framework's reusable portions. The left side shows reusable logic implemented by the QExpress framework. The right side shows abstraction upon hardware and computation state utilized by the framework.

3.4 Hardware Specific Extensions

Hardware extensions provide additional information to the compiler framework. As alluded to previously, this portion handles specifics that can not be separated from an intimate knowledge of the target hardware. Great effort was made to elevate as much logic as possible above this level in order to increase code reusability and decrease the amount of effort needed to implement a custom compiler. As a consequence, the reusable portions of the QExpress framework expect only two methods to be implemented by hardware extensions: the building of adjacency lists and the validation of qubit routing paths. The following sections will detail how these two methods are implemented for three example hardware extension implementations, one each for neutral atom, trapped ion, and superconducting systems.

3.4.1 Adjacency List Creation

The adjacency list expected by the QExpress framework should consist of nodes representing hardware sites capable of holding a qubit for any reason (including storage, transport, interaction) and edges representing possible paths of interaction or movement. Sites can be marked as any combination of compute, storage, or transport, reflecting what actions they may be used for during the compute process. While exact hardware implementations are expected to vary for different computers, there are still generalizations to be made. All systems of the same architectural class tend to behave somewhat similarly when dealing with qubit movement and interaction. Therefore, for each architecture, a few general rules can be established for building the adjacency lists.

Neutral Atom

A neutral atom computer is composed of both static and mobile arrays of qubits. It is often a zoned architecture. Spatial Light Modulators (SLMs) arrays create a static grid of traps and Acousto-Optic Deflectors (AODs) create motile grids which can be spatially translated and sometimes reshaped or rotated. Motion of AODs are continuous, but they often must satisfy some constraints such as never allowing rows and columns within an array to cross. Interaction of qubits is done by placing qubits within a Rydberg radius and enabling a global entanglement pulse. Qubits that are not to be interacted must not be within Rydberg radius of each other when the entanglement pulse is activated. As a result of these peculiarity, neutral atom systems are potentially incredibly capable but have the most complex adjacency lists. Given a listing of SLM and AOD arrays as well as a Rydberg radius, an adjacency list can be constructed by the following rules:

- All sites within compute zones of the SLMs are marked as compute sites. All sites within the storage zones of the SLMs are marked as storage sites. All sites within the AODs are marked as transport sites.
- All sites within an array are not connected to each other, with one exception: each compute site $\text{HW_}si, i \in \mathbb{N}$ has a corresponding compute site $\text{HW_}si_r$ and $\text{HW_}si$ and $\text{HW_}si_r$ are connected.
- All storage and compute sites are fully connected to all transport sites.

Figure 3.7 demonstrate an application of these rules to a simple system.

Trapped Ion

A trapped ion computer is composed of traps and shuttles between traps. It is often implicitly zoned with traps serving as both storage and compute zones and shuttles serving as transport zones. Within a trap, qubits are able to be moved past each other and arbitrarily pair-wise interacted. Shuttles allow for movement of qubits from one trap to another trap. Given a listing of traps, shuttles, and sites within, an adjacency list can be constructed by the following rules:

- All sites within traps are marked as compute and storage sites. All sites within shuttles are marked as transport sites.

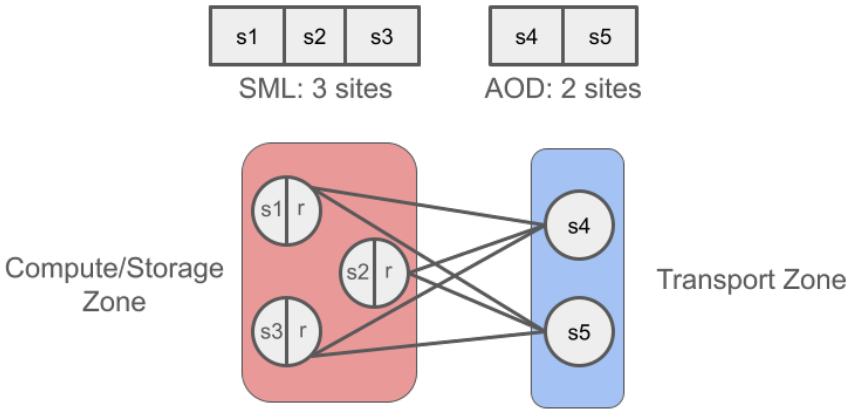


Figure 3.7: Above is a simple neutral atom system with one SLM containing 3 sites and an AOD containing 2 sites. This SLM is designated as both a compute and storage zone. Below is the corresponding adjacency list representation.

- All sites within a trap are fully connected to each other.
- All sites within a shuttle are fully connected to each other.
- Each shuttle site connects to every site in each trap in its corresponding traps.

Figure 3.8 demonstrates an application of these rules to a simple system.

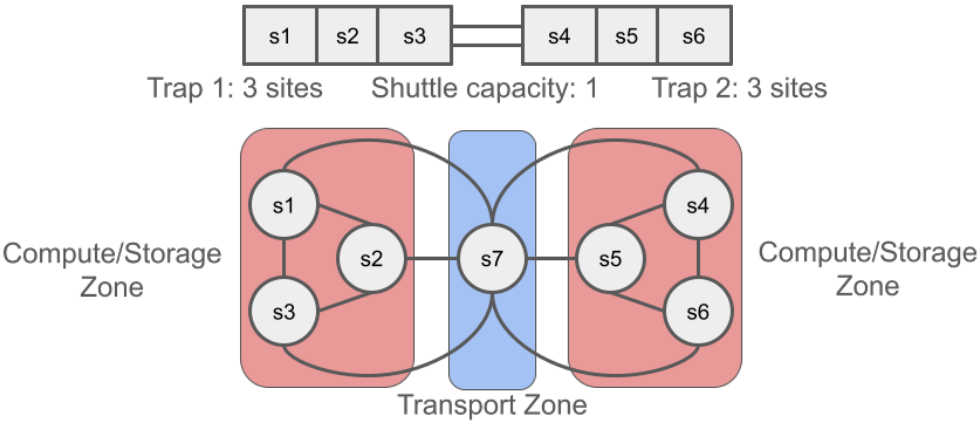


Figure 3.8: Above is a simple trapped ion system with two traps, each contain two sites and a shuttle with capacity 1 between them. Below is the corresponding adjacency list representation.

Superconducting

A superconducting computer is composed of many Josephine junctions or other such implementation of qubits. It is often a non-zoned. The qubits are arranged in a set connectivity pattern by physical conduits. Qubits adjacent on the connectivity topology can be swapped[2] or pairwise interacted. Given a description of the

[2] As mentioned before, this is an exchange of qubit state. The physical qubits remain immobile, unlike neutral atom or trapped ion systems

connectivity, the adjacency list can trivially be constructed[3]:

- All sites are marked as compute, storage, and transport sites.
- Sites are connected exactly as they are in the hardware connectivity graph.

Figure 3.9 demonstrates an application of these rules to a simple system.

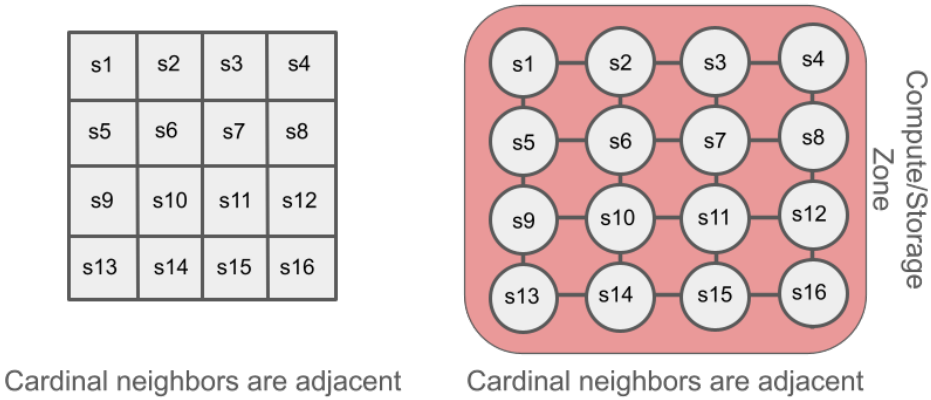


Figure 3.9: Left is a simple superconducting system containing 16 sites. Qubits are connected in cardinal directions. Right is the corresponding adjacency list representation.

3.4.2 Process Path

The process path method provides and opportunity to enforce any specific hardware constraints on movement which may be present on the target system and can not be captured by the adjacency list. Again, while implementation details are expected to differ between architectures, but there are general rules that are followed for all architectures.

Neutral Atom

All movement on Neutral atom systems are based on AODs. Additionally, AODs are typically able to traverse the entire compute surface. Therefore, valid movement paths must begin with either a compute or storage site, pass through a transport site, and return to either a compute or storage site. All sites in the movement sequence must be unoccupied except the first site as that is the qubit to be moved.

Trapped Ion

Movement in trapped ion systems are based on shuttling ions. This results in a similar profile to neutral atom movements except the total path may exceed three sites. A valid route must start and end in a compute or storage site. It must also alternate between storage or compute and transport sites. Again, sites in the movement sequence except the first must be unoccupied.

[3]This leads to simplicity of debugging later on, and the author really likes this architecture.

Superconducting

Movement is swap based between adjacent qubits. A valid route simply has to pass through a valid path on the hardware connectivity graph. As the adjacency list is a representation of the hardware connectivity, any valid path through the adjacency list is acceptable. Unlike neutral atom and trapped ion systems, sites within a superconducting route may be occupied or unoccupied.

Using only the adjacency list and site labels, most movement restrictions can be enforced in a simple manner. More complex rules, for example regarding parallelized movements using a single AOD on neutral atom systems, can be added to the process route method as needed, but in many instances, the above should suffice.

It is of note that the implemented hardware extension files are fairly generalizable. In combination with the proper hardware information provided at compile time, the three implemented hardware extension should be suitable for most systems in each of the architectural families without modification. Especially unique hardware within an architectural family which break the rules described above for adjacency list creation and path processing will however require a customized hardware extension. In such cases, these extensions should still be a serviceable guide on how to implement a custom extension. The following subsection shows how these responsibilities appear in emitted hardware code.

3.4.3 Hardware Code Realization

Having established how adjacency and path processing are defined, this section describes how those choices are realized in emitted hardware code. Hardware specific behavior is isolated behind a narrow interface in the back end. The compiler entry point selects the proper back end implementation based on `target_arch` and then invokes a shared compilation flow for instruction traversal, initial placement, and routing heuristic selection. Hardware independent logic remains in the base back end, while each hardware extension overrides only architecture specific responsibilities.

Concretely, each hardware extension contributes two core implementations: first, construction of the hardware adjacency list and zone labels from the architecture configuration, and second, processing of selected movement routes. The overall control flow is shared, but emitted movement semantics differ by architecture. Neutral atom compilation emits AOD mediated movement operations (for example, `MOVE_AOD` and relocation operations), trapped ion compilation emits shuttle based operations (`Move_Shuttle` and `FireShuttle`), and superconducting compilation emits swap chain operations (`Move_Swap`) over the physical coupling graph. In this way, the QExpress framework reuses hardware aware placement and routing logic while localizing architecture specific assumptions to compact hardware extension files.

To illustrate emitted hardware instructions, the snippet below shows a short excerpt from trapped ion output. The sequence demonstrates the expected pattern: map qubits to hardware sites, shuttle through transport links, and apply two qubit gates once adjacency is achieved.

```
NewQubit q1 = new Qubit(HW_s0);
```

```
NewQubit q3 = new Qubit(HW_s1);
Move_Shuttle(HW_s3, HW_s8)
FireShuttle(shuttle2)
Move_Shuttle(HW_s8, HW_s5)
TrappedIon.applyOperation(sites = [['HW_s5', 'HW_s6']],
                          gate_name = 'rxx',
                          parameters = ['pi/2'])
```

Code Snippet: Excerpt of trapped ion hardware code generated by the back end, showing qubit initialization, shuttle movement, and a two qubit `rxx` interaction once qubits are adjacent. This sequence can be reproduced with the trapped-ion blockage movement test case included in the repository.

The exact emitted hardware code format can be customized by the user to suit the needs of a specific system. For example, architectures with special capabilities may require additional instruction types beyond those shown above. The caveat is that the provided back end optimization and analysis passes may not operate correctly on newly introduced instructions without corresponding pass updates.

This concludes the description of the QExpress compiler framework. An overview of the implemented logic and abstractions can be seen in **Figure 3.10**. Additional implementation details can be found in the **Appendices A.1**.

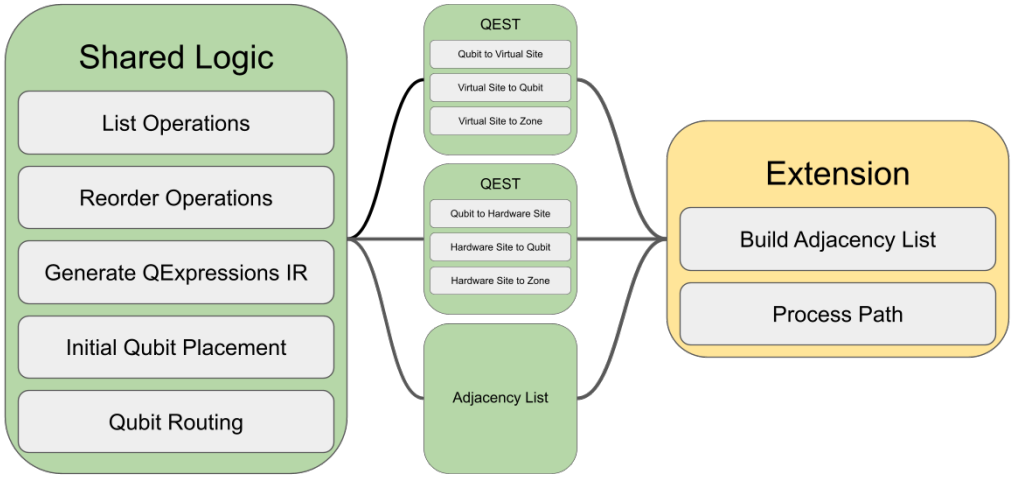


Figure 3.10: A view of the completed QExpress framework. Any complete compiler implemented using the framework need only implement the "Extension" section shown in yellow.

Chapter 4

Compiler Overview

4.1 A QExpress Compiler

The following section and future sections concern the use of the QExpress framework in building compilers. Compilers built with the framework require two inputs: a quantum circuit specifying the circuit to be compiled and a JSON file detailing some aspects of the hardware. Currently, the QExpress framework provides support for input for circuits in QASM 2.0 due to it's ubiquitous use in the field. A JSON schema is included in the QExpress framework to demonstrate what hardware information is necessary and how it should be formatted. Inputs are further detailed in **Section 4.3**.

Figure 4.1 shows a high-level overview of the compilation process for the QExpress compiler. The user must provide the circuit to be compiled as a QASM file and a description of the hardware in a JSON format adhering to our hardware configuration JSON Schema. There are two oppotunities for optimization, one at the QExpression level and one at the hardware code level. The QExpression level optimizations are hardware agnostic while the hardware code level optimizations are hardware aware. Optimizations at the QExpression level are inherently valid for all architectures while optimizations at the hardware code level may or may not be valid for multiple architectures. Each optimization is implemented as distinct optimization pass, and the optimization stages are made up of multiple passes. Passes are controlled by a pass manager which is responsible for enabling and disabling each optimization pass as per user instruction.

Hardware builders are responsible for providing the hardware extension file as described in **Chapter 3** as they have detailed knowledge of what physical actions must be taken to affect the logical instructions prescribed by each QExpression. The existing implemented extensions provide many reusable sub-routines and utilities as well as concrete examples to aid in this development. This computation level logic is meant to remove the burden of hardware knowledge currently borne by developers aiming to implement a quantum program on a target system. The QExpress framework includes three hardware extension, and therefore three compilers, already implemented.

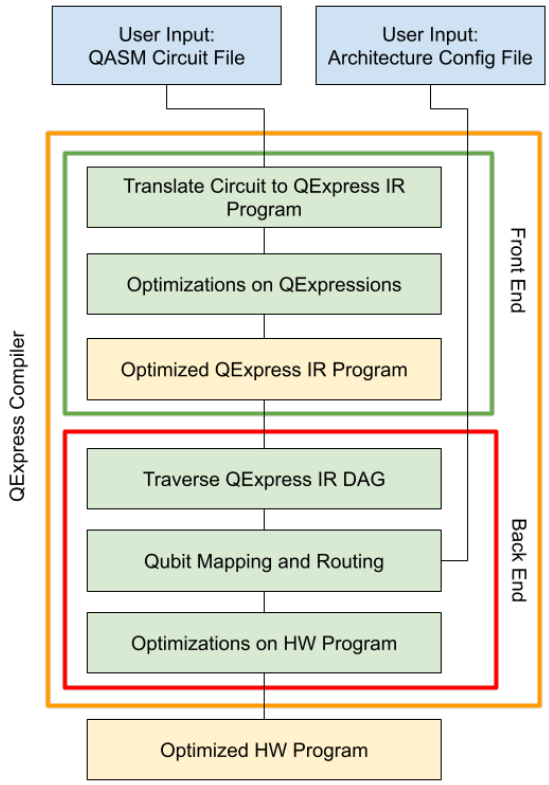


Figure 4.1: A view of a compiler implemented using the QExpress framework showing inputs, abbreviated steps, intermediate results, and outputs.

4.2 The Pass Manager

After the main goal of translation, optimization is the primary goal of modern compilers. It is possible to build optimizations directly into the compilation flow, but this may lead to difficulties including: difficulty to debug, incompatibility between optimizations, and reduced portability among others. To pre-empt these shortfalls, the QExpress framework implements optimizations in **passes** and a robust **Pass Manager** which controls which optimizations are run and in what order at the user's instruction.

Distinct optimizations are implemented in distinct code files, and they all inherit from the **BasePass** class and therefore have a similar structure. The BasePass class defines a rigid interface with the pass manager as well as many useful subroutines which are likely to be required by many optimizations. Each instance of the class also has it's own QEST instance which can be initialized from the saved state of other QEST instances. Finally, the class implements rigid structure to the inputs and outputs of optimization passes. Therefore as long as prescribed interfaces are maintained, passes can be added, removed, reordered, or edited as desired.

Passes can be either analysis or transformation passes with the distinction being that analysis passes generate new data but do not modify the compilation output while transformation passes do alter the compilation output. Both are important as the same analysis pass often produces the information necessary to carry out many different transformative optimizations. The ability to separate analysis logic from transformation logic reduces code repetition. To ease inter-pass information

propagation, passes also have the ability to write any metadata they wish to the compilation output in specified metadata sections. This can include simple analysis information or even preserved QEST states to be consumed by subsequent passes.

The pass manager is called upon twice; once at the end of QExpression generation in the compiler front end, and once after hardware code is generated. Some passes are exceedingly general (such as an analysis pass counting the number of movements) and can be used for all architectures and both the front and back ends. However, the pass manager provides the developer with the option of writing highly specific passes that may only be invoked at either the front or back end and for a select set of architecture or even one specific hardware.

A few simple analysis and transformation passes have been implemented in the QExpress framework including:

- Analysis: Timing, fidelity, DAG Parallelization
- Transform: Store qubit after last use, remove unused sites, order moves by degree

Developers are encouraged to implement new optimizations as they need. Current work is being done to expand the set of optimizations currently included in the QExpress framework.

4.3 Compiler Usage

Using a QExpress compiler is fairly straight forward. The user should provide a circuit specified in OpenQASM 2.0 format and a hardware configuration adhereing to the provided JSON schema detailing what information is necessary for each architecture. This information regards the layout of the compute surface, the native gate set, and values necessary for computing timing and fidelity estimates. Command line options allow the user to control almost all aspects of the compiler, including what heuristic algorithms are used for qubit placement or routing and what optimization passes and in what order they are to be run. It should be noted that QExpress does not support functionality specific to OpenQASM as maintained by IBM. OpenQASM is used only as a language to specify circuit logic. If the user wishes to use a different language, they must re-write the operations parser which was described in **Section 3.2.1**.

The compiler generates a text file containing hardware instructions and metadata as output and also prints this output to the console. A command line option is available to specify specific input and output files. The first output is the optimized QExpressions followed by metadata of any passes run on them. Next is the optimized hardware code and the metadata of passes run on them. For the purposes of debugging, the passes can be instructed to print out the instructions they are operating upon both before and after the pass is run. This can be especially helpful for transformation passes.

Examples of commands, inputs, and outputs can be found in the **Appendices A.2**.

Chapter 5

Results and Further Work

5.1 Framework Evaluation

The main goal of the QExpress project is to develop a highly portable compiler framework for quantum computing that can be used to build compilers for specific hardware while reusing as much compiler logic as possible. In order to demonstrate the framework, three hardware extensions were implemented, one each for neutral atom, trapped ion, and superconducting systems. **Table 5.1** shows the complexity of the QExpress framework as well as the hardware extension files in terms of number of distinct code files and total lines of code. The distinction between what is considered framework and what is considered an extension is consistent with descriptions for the rest of the paper.

It is clear to see that a majority of the complexity resides within the framework itself. As described in **Chapter 3**, much of the compiler logic is shared between architectures, even for back end tasks such as qubit placement and routing which in previous work has been often treated as intrinsically linked to one architectural family if not one particular system[4, 11, 10, 5, 16].

Therefore, QExpress succeeds in its goal as a modular framework for building quantum compilers. Ongoing work is concerned with determining how well QExpress compilers compare against bespoke compilers for the same system.

Implementation	**Number of Files**	**Total Lines of Code**
Framework	22	5168
Trapped Ion	1	107
Superconducting	1	35
Neutral Atom	1	194

Table 5.1: Summary of framework vs hardware extension code complexity. Most compilation logic is contained within the framework. The listings for each architecture are the statistics for their respective hardware extension file. Neutral atom has the most complex movement and connectivity scheme, and therefore requires the most code. Superconducting systems by comparison require very few details to be handled in the hardware extension portion as most relevant information can directly be gleaned from the hardware connectivity of a system.

5.1.1 Unblocking

Algorithm 3 Blockage Resolution

Require: qubits q_1, q_2 that must interact
$A_1 \leftarrow$ compute neighbors of q_1 excluding $\{q_1, q_2\}$
$A_2 \leftarrow$ compute neighbors of q_2 excluding $\{q_1, q_2\}$
$C \leftarrow \emptyset$ {candidate endpoints}
for each $a \in A_1 \cup A_2$ **do**
  if $a \in A_1$ **then**
    $p \leftarrow$ BFS path from q_2 to a allowing occupied sites
  else
    $p \leftarrow$ BFS path from q_1 to a allowing occupied sites
  end if
  if $p \neq \emptyset$ **then**
    add (a, p) to C
  end if
end for
if $C = \emptyset$ **then**
  return None
end if
for each $(a, p) \in C$ **do**
  $proximity(a) \leftarrow |p|$
  $futureScore(a) \leftarrow$ future interaction score of a
  $totalScore(a) \leftarrow proximity(a) + futureScore(a)$
end for
$a^* \leftarrow \arg\min_{a \in C} totalScore(a)$
if a^* is $\emptyset$ **then**
  return None
end if
$q_b \leftarrow$ qubit occupying a^*
$dest \leftarrow$ closest free compute site to a^*
if $dest$ is $\emptyset$ **then**
  return None
end if
move q_b from a^* to $dest$
return $(q_b, a^*, dest)$

During computation on trapped ion systems, it is natural for a trap to become full. When this occurs, all shuttling into that trap cannot occur, and therefore shuttling through the trap cannot occur. This phenomenon is resolved with "unblocking" algorithms that re-locate some qubits from the trap at max capacity to other nearby traps so as to ensure the flow of qubits may resume.

In order to address this issue in QExpress, a dedicated unblocking algorithm was implemented, and in the spirit of the project, the algorithm was written at a generalized level. It was quickly noticed that this algorithm had unexpected uses outside of compiling for the trapped ion architecture.

Neutral atom systems experience "Rydberg blockading" when qubits unintended to interact are within Rydberg radius of each other. Superconducting systems ex-

perience "cross-talk" whereby interactions between two qubits can affect the state of nearby, uninvolved qubits. Under the previously described abstractions utilized in the qubit routing process, all three problems reduce to the same problem: unblocking.

The same unblocking algorithm originally intended for use specifically for trapped ion systems could be used, without modification, to resolve seemingly unrelated problems on other architectures. **Algorithm 3** shows the unblock implementation included in QExpress.

In hindsight, it is clear that these problems across architectures are all related, but at a cursory inspection these problems arise from fundamentally different mechanisms and there is not much indication of their relation. It is possible that there are many other such instances of problems across architectures which seem unrelated but in actuality have the same solution. It is hoped that the generalizable nature of QExpress can allow for the discoveries of more such higher order symmetries and solutions.

5.2 Further Work

Current work on the QExpress framework can be divided into two approaches: benchmarking and further expansion of capabilities.

Benchmarking compilers developed using the QExpress framework against bespoke compilers for a given system will show whether the framework can produce good enough results to be relevant as more than a proof of concept. Initial results are promising and show QExpress compilers performing on par or better than bespoke compilers in number of moves generated and final circuit fidelity though additional testing is needed before such claims can be made with certainty.

Part of the difficulty in such an endeavor is the lack of standard benchmarking suites. As such, one part of the effort is to curate a broad test suite for all three architectures currently supported by the framework. A second point of difficulty comes from finding bespoke compiler implementations. Many papers on compilation exist, though many of them do not have source code easily accessible. To give the best chance for a fair comparison, it is best that code written by the original compiler implementers can be used. Therefore, effort is being made to catalog quantum compiler implementations which are accessible for benchmarking.

On the front of extending capabilities, two areas of focus are writing additional optimization passes and extending the framework to more architectures. Circuit optimizations are a common approach to improve final output fidelity. QExpress was designed with the eventual writing of such passes in mind, and so the work to do just that commences. There are many such optimizations possible, and most if not all of them are conducive to pass based structure that QExpress uses. It is hoped that in the future, there will be a library of preimplemented optimizations that developers can pick and choose from.

Extension of the QExpress system to other emerging quantum computing architectures can boost the likelihood of adoption for the framework by the quantum compilation development community. Therefore, extending the framework to include support for photonic systems is desired.

Chapter 6

Conclusion

In summary, the QExpress compiler framework provides a flexible and extensible foundation for the rapid development of optimizing compilers targeting a wide range of quantum hardware platforms. Currently, neutral atom, trapped ion, and superconducting architectures are supported, but extensions to photonic and other gate based architectures are possible. By separating the compilation pipeline into hardware independent and hardware aware components, corresponding to the front and back ends, the framework enables much of the compilation logic to be implemented in a generalizable and reusable manner. This design significantly reduces the need for architecture-specific implementations and promotes portability across diverse hardware targets.

Central to the framework is the QExpressions intermediate representation, which captures quantum circuit logic in a form that exposes computational structure through the instruction set of the QExpress Quantum Virtual Machine. Complementing this representation is a comprehensive analysis and optimization pass management system, supporting the development and application of portable optimizations at both the intermediate representation level and in the final hardware-specific code. As a result, implementing a compiler for a new quantum system is streamlined; developers may leverage existing hardware extensions, adapting them as needed to accommodate the characteristics of the target architecture. Together, these contributions demonstrate how QExpress enables efficient, modular, and reusable compiler development for emerging quantum computing platforms.

Appendix A

QExpress Implementation Details

A.1 QExpress Code Overview

Code for the latest release can be found at:

`https://github.com/QExpressCompiler/QExpress/releases/latest`

A.1.1 Front End

At a high level, the control flow begins in `QEXPCompile.py` and then enters the front end implementation in `FrontEnds/unifiedFE.py`. Together, these files capture how an input circuit is transformed into executable instructions.

The front end is responsible for reading circuit operations, optionally reordering them, and emitting hardware agnostic `QEXP_VM` instructions, referred to in this thesis as QExpressions. In practice, this means the front end builds the virtual machine regions and sites, maps logical operations onto VM actions, and emits the instructions consumed by the back end.

Key front end files include:

- `QEXPCompile.py`: pipeline entry point and process control.
- `FrontEnds/unifiedFE.py`: primary front end implementation.
- `FrontEnds/reorderOperations.py`: operation ordering methods (for example, `none`, `topoSort`).
- `FrontEnds/generateInstructions.py`: VM movement and operation generation strategies.
- `utils/ParseInput.py`: circuit and configuration parsing.

A.1.2 Back End

After front end generation, the compiler translates QExpressions into hardware instructions in the back end. Shared back end behavior lives in `BackEnds/BackEnd.py`, while architecture-specific movement and connectivity logic is implemented in one of three extension files.

Key back end files include:

- `BackEnds/BackEnd.py` for shared translation, batching, and state tracking.
- `BackEnds/neutralAtomBE.py` for neutral atom hardware translation.
- `BackEnds/trappedIonBE.py` for trapped ion hardware translation.
- `BackEnds/superCondBE.py` for superconducting hardware translation.

Each hardware extension primarily defines how adjacency is built and how qubit movement routes are realized on that architecture.

A.1.3 Optimization & Analysis

QExpress runs optimization and analysis passes in two places: once after front end generation and again after back end translation. These passes are managed through a common pass manager interface and are appended to output metadata in the final `.qexp` file.

This stage addresses three questions: what can be simplified, what can run in parallel, and what are the expected timing and fidelity estimates. Core files are:

- `Optimizations/BasePass.py`: shared pass interface.
- `Optimizations/removeUnusedSites.py`, `storeInactive.py`: front end cleanups.
- `Optimizations/DAGParallelization.py`: parallelization analysis.
- `Optimizations/timing.py`: runtime estimation.
- `Optimizations/fidelity.py`: fidelity estimation.

The compilation process is therefore: parse inputs, generate QExpressions, run FE passes, translate to hardware instructions, run BE analyses, and write the final output sections.

A.2 Examples

The QExpress framework includes three hardware extensions (and therefore three compilers). A common command pattern is:

```
./.venv/bin/python QEXPCompile.py
[target_arch=xxx] [circuit_file=xxx] [reorder_ops=xxx]
[vm_connectivity=xxx] [initial_placement=xxx] [movement_heuristic=xxx]
[opt_passes=xxx,xxx,xxx] [config_file=xxx] [output_file=xxx]
```

The main command line options are:

- `[target_arch]` takes one of `neutral_atom`, `trapped_ion`, or `super_cond`
- `[circuit_file]` specifies the QASM circuit to compile
- `[reorder_ops]` specifies an algorithm for reordering operations
- `[vm_connectivity]` specifies Quantum VM connectivity

- [initial_placement] specifies the initial hardware placement heuristic
- [movement_heuristic] specifies the heuristic for qubit movement
- [opt_passes] lists optimization passes, separated with commas
- [config_file] specifies a hardware JSON document conforming to schema.json
- [output_file] defaults to superCondOutput.qexp, neutralAtomOutput.qexp, or trappedIonOutput.qexp

A.2.1 Neutral Atom Example

This neutral atom example emphasizes the full movement model:

```
./.venv/bin/python QEXPCompile.py target_arch=neutral_atom
circuit_file=inputs/circuits/ghz_with_cz.txt reorder_ops=topoSort
vm_connectivity=allToAllComputeMove
opt_passes=removeUnusedSites,DAGParallelization,timing,fidelity
config_file=inputs/neutralConfigs/neutral_hardware.json
output_file=neutralAtomOutput.qexp
```

Inputs:

- Circuit file: inputs/circuits/ghz_with_cz.txt
- Hardware config file: inputs/neutralConfigs/neutral_hardware.json

Sample hardware configuration snippet (abridged):

```
{
    "Size": [100, 100],
    "NumQubits": 100,
    "RydbergRadius": 0.1,
    "Arrays": {
        "AOD": {
            "type": ["transport"],
            "speed": 55,
            "Rows": [0, 10, 20],
            "Cols": [0, 10, 20],
            "capacity": 36
        },
        "SLM": {
            "type": ["storage", "compute"],
            "Rows": [0, 10, 20],
            "Cols": [0, 10, 20],
            "capacity": 100
        }
    },
    "Gates": {
        "h": {
            "NumberOfParameters": 0,
```

```
            "NumberOfQubits": 1,
            "ExecutionTime": 2,
            "ErrorRate": 0.000127
        },
        "cz": {
            "NumberOfParameters": 0,
            "NumberOfQubits": 2,
            "ExecutionTime": 0.8,
            "ErrorRate": 0.0048
        }
    }
}
```

In this configuration, `Arrays` and `RydbergRadius` define the geometric model, including storage/compute placement and interaction locality. The `Gates` section records operation arity, parameter count, execution time, and error rate values that feed timing and fidelity analysis.

Circuit snippet (from the input file):

```
OPENQASM 2.0;
qreg q[3];
h q[0];
cz q[0],q[1];
cz q[1],q[2];
```

Output:

- A generated file named `neutralAtomOutput.qexp`

Sample output snippet (abbreviated):

```
MOVE(s0, s3)
QEXP_VM.applyOperation(sites = [s3], gate_name = 'h', parameters = [])
MOVE_AOD(HW_s2, HW_s8)
RelocateAOD(('AOD', 'HW_s2', 'HW_s5r'))
NeutralAtom.applyOperation(sites = [HW_s5, HW_s5r], gate_name = 'cz')
```

A.2.2 Trapped Ion Example

This trapped ion example highlights shuttle based transport across trap regions:

```
./.venv/bin/python QEXPCompile.py target_arch=trapped_ion
circuit_file=inputs/circuits/ghz_with_rxx.txt reorder_ops=topoSort
vm_connectivity=allToAllComputeMove
opt_passes=removeUnusedSites,DAGParallelization,timing,fidelity
config_file=inputs/trappedIonConfigs/trapped_ion_config.json
output_file=trappedIonOutput.qexp
```

Inputs:

- Circuit file: `inputs/circuits/ghz_with_rxx.txt`
- Hardware config file: `inputs/trappedIonConfigs/trapped_ion_config.json`

Sample hardware configuration snippet (abridged):

```
{
    "NumQubits": 9,
    "OpZones": ["MemoryZone", "InteractionZone"],
    "TimetoSwitchTraps": 100,
    "Arrays": {
        "Trap1": {"type": ["storage", "compute"], "capacity": 2},
        "Trap2": {"type": ["storage", "compute"], "capacity": 2},
        "shuttle1": {
            "type": ["transport"],
            "traps": ["Trap1", "Trap2"],
            "capacity": 1
        }
    },
    "Gates": {
        "h": {
            "NumberOfParameters": 0,
            "NumberOfQubits": 1,
            "ExecutionTime": 2,
            "ErrorRate": 0.0005
        },
        "rxx": {
            "NumberOfParameters": 1,
            "NumberOfQubits": 2,
            "ExecutionTime": 2,
            "ErrorRate": 0.005
        }
    }
}
```

In this configuration, `Arrays` separates stationary traps from transport shuttles, with explicit shuttle to trap links in `traps`. `OpZones` and `TimetoSwitchTraps` capture interaction region behavior and reconfiguration overhead, while `Gates` defines native operations (for example, `rxx`) and associated timing/error metadata.

Circuit snippet (from the input file):

```
OPENQASM 2.0;
qreg q[3];
h q[0];
rxx(1.5708) q[0],q[1];
rxx(1.5708) q[1],q[2];
```

Output:

- A generated file named `trappedIonOutput.qexp`

Sample output snippet (abbreviated):

```
MOVE(s0, s3)
QEXP_VM.applyOperation(sites = [s3], gate_name = 'h', parameters = [])
```

```
Move_Shuttle(HW_s1, HW_s6)
FireShuttle(ShuttleA)
TrappedIon.applyOperation(sites = [HW_s4, HW_s5], gate_name = 'rxx')
```

A.2.3 Superconducting Example

This superconducting example highlights graph based routing over fixed couplers:

```
./.venv/bin/python QEXPCompile.py target_arch=super_cond
circuit_file=inputs/circuits/ghz.txt reorder_ops=topoSort
vm_connectivity=allToAllComputeMove initial_placement=random
opt_passes=removeUnusedSites,DAGParallelization,timing,fidelity
config_file=inputs/supercondConfigs/supercond_config_tiny.json
output_file=superCondOutput.qexp
```

Inputs:

- Circuit file: `inputs/circuits/ghz.txt`
- Hardware config file: `inputs/supercondConfigs/supercond_config_tiny.json`

Sample hardware configuration snippet (abridged):

```
{
    "EdgeList": [[0,1],[1,2],[2,3],[3,4]],
    "Arrays": {
        "QubitArray1": {
            "type": ["storage", "compute"],
            "capacity": 5
        }
    },
    "Gates": {
        "h": {
            "NumberOfParameters": 0,
            "NumberOfQubits": 1,
            "ExecutionTime": 0.2,
            "ErrorRate": 0.0005
        },
        "cx": {
            "NumberOfParameters": 0,
            "NumberOfQubits": 2,
            "ExecutionTime": 0.2,
            "ErrorRate": 0.015
        },
        "SWAP": {
            "NumberOfParameters": 0,
            "NumberOfQubits": 2,
            "ExecutionTime": 0.6,
            "ErrorRate": 0.003
        }
    }
}
```

In this configuration, `EdgeList` encodes the coupling graph used for routing and swap insertion, while `Arrays` defines aggregate device capacity. `Gates` captures native operations such as `cx` and `SWAP`, along with execution time and error rate values consumed by timing and fidelity passes.

Circuit snippet (from the input file):

```
OPENQASM 2.0;
qreg q[3];
h q[0];
cx q[0],q[1];
cx q[1],q[2];
```

Output:

- A generated file named `superCondOutput.qexp`

Sample output snippet (abbreviated):

```
MOVE(s0, s3)
QEXP_VM.applyOperation(sites = [s3], gate_name = 'h', parameters = [])
Move_Swap(HW_s0, HW_s1)
Move_Swap(HW_s1, HW_s2)
SuperCond.applyOperation(sites = [HW_s2, HW_s3], gate_name = 'cx')
```

References

[1] Chenghong Zhu, Xian Wu, Zhaohui Yang, Jingbo Wang, Anbang Wu, Shenggen Zheng, and Xin Wang. Quantum Compiler Design for Qubit Mapping and Routing: A Cross-Architectural Survey of Superconducting, Trapped-Ion, and Neutral Atom Systems, May 2025. arXiv:2505.16891 [quant-ph].

[2] Jixuan Ruan, Xiang Fang, Hezi Zhang, Ang Li, Travis Humble, and Yufei Ding. PowerMove: Optimizing Compilation for Neutral Atom Quantum Computers with Zoned Architecture, November 2024. arXiv:2411.12263 [quant-ph].

[3] Abdullah Ash Saki, Rasit Onur Topaloglu, and Swaroop Ghosh. Muzzle the Shuttle: Efficient Compilation for Multi-Trap Trapped-Ion Quantum Computers. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 322–327, March 2022. arXiv:2111.07961 [quant-ph].

[4] Henry Zou, Matthew Treinish, Kevin Hartman, Alexander Ivrii, and Jake Lishman. LightSABRE: A Lightweight and Enhanced SABRE Algorithm, September 2024. arXiv:2409.08368 [quant-ph].

[5] Hanrui Wang, Daniel Bochen Tan, Pengyu Liu, Yilian Liu, Jiaqi Gu, Jason Cong, and Song Han. Q-Pilot: Field Programmable Qubit Array Compilation with Flying Ancillas, September 2024. arXiv:2311.16190 [quant-ph].

[6] Prakash Murali, Norbert Matthias Linke, Margaret Martonosi, Ali Javadi Abhari, Nhung Hong Nguyen, and Cinthia Huerta Alderete. Full-stack, real-system quantum computer studies: architectural comparisons and design insights. In *Proceedings of the 46th International Symposium on Computer Architecture*, pages 527–540, Phoenix Arizona, June 2019. ACM.

[7] Fabian Kreppel, Christian Melzer, Diego Olvera Millán, Janis Wagner, Janine Hilder, Ulrich Poschinger, Ferdinand Schmidt-Kaler, and André Brinkmann. Quantum Circuit Compiler for a Shuttling-Based Trapped-Ion Quantum Computer. *Quantum*, 7:1176, November 2023. arXiv:2207.01964 [quant-ph].

[8] Z. T. Wang, Qiuhao Chen, Yuxuan Du, Z. H. Yang, Xiaoxia Cai, Kaixuan Huang, Jingning Zhang, Kai Xu, Jun Du, Yinan Li, Yuling Jiao, Xingyao Wu, Wu Liu, Xiliang Lu, Huikai Xu, Yirong Jin, Ruixia Wang, Haifeng Yu, and S. P. Zhao. Quantum Compiling with Reinforcement Learning on a Superconducting Processor, June 2024. arXiv:2406.12195 [quant-ph].

[9] Gushu Li, Yufei Ding, and Yuan Xie. Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices, May 2019. arXiv:1809.02573 [cs].

[10] Yannick Stade, Wan-Hsuan Lin, Jason Cong, and Robert Wille. Routing-Aware Placement for Zoned Neutral Atom-based Quantum Computing, May 2025. arXiv:2505.22715 [quant-ph].

[11] Jason Ludmir and Tirthak Patel. Parallax: A Compiler for Neutral Atom Quantum Computers under Hardware Constraints, September 2024. arXiv:2409.04578 [quant-ph].

[12] Hanrui Wang, Pengyu Liu, Daniel Bochen Tan, Yilian Liu, Jiaqi Gu, David Z. Pan, Jason Cong, Umut A. Acar, and Song Han. Atomique: A Quantum Compiler for Reconfigurable Neutral Atom Arrays, May 2024. arXiv:2311.15123 [quant-ph].

[13] Dmitri Maslov. Basic circuit compilation techniques for an ion-trap quantum machine. *New Journal of Physics*, 19(2):023035, February 2017. arXiv:1603.07678 [quant-ph].

[14] Daniel Bochen Tan, Wan-Hsuan Lin, and Jason Cong. Compilation for Dynamically Field-Programmable Qubit Arrays with Efficient and Provably Near-Optimal Scheduling, May 2024. arXiv:2405.15095 [quant-ph].

[15] Zebo Yang, Maede Zolanvari, and Raj Jain. A Survey of Important Issues in Quantum Computing and Communications. *IEEE Communications Surveys & Tutorials*, 25(2):1059–1094, 2023.

[16] Daniel Bochen Tan, Dolev Bluvstein, Mikhail D. Lukin, and Jason Cong. Compiling Quantum Circuits for Dynamically Field-Programmable Neutral Atoms Array Processors. *Quantum*, 8:1281, March 2024. arXiv:2306.03487 [quant-ph].